

0.107.1. Une lettre qui tire — une explosion plus grande

Le 0.107, mais les bras de la croix ont une longueur, PORTEE = 2 : une boucle for les allume case après case. Cinq X cassés au lieu de trois.

1. Ce qu'il faut comprendre

C'est le 0.107, avec une explosion plus grande. Chaque bras de la croix a maintenant **deux** cases : l'explosion casse cinq X dans le mur, des lignes 6 à 10.

Le problème : au 0.107, `explorer()` écrivait les cinq cases une par une. Avec des bras de deux cases, il en faudrait neuf ; de trois, treize... Et autant dans `eteindreTout()`.

Ce qui est nouveau ici : une boucle pour les bras. `PORTEE` dit la longueur d'un bras. Après le centre, `for (uint8_t k = 1; k <= PORTEE; k++)` : `k` vaut 1, puis 2. À chaque tour, les **quatre** bras gagnent une case : `(ex + k, ey)`, `(ex - k, ey)`, `(ex, ey + k)`, `(ex, ey - k)`. `<=` veut dire « plus petit ou égal » : avec `PORTEE = 2`, le tour `k = 2` se fait aussi.

`eteindreTout()` **fait la même boucle**, avec `eteindre()` à la place d'`allumer()`. Les deux passent sur les mêmes cases.

Au bord de l'écran : `ex - 2` peut donner 254 ou 255 ; `allumer()` et `eteindre()` le refusent déjà (le 0.107).

Déroulons : la boule touche (12, 8). Le centre, puis `k = 1` : (13, 8), (11, 8), (12, 9), (12, 7). `k = 2` : (14, 8), (10, 8), (12, 10), (12, 6). Neuf « # », cinq X cassés.

Essaie : `PORTEE = 4`, et regarde la croix traverser presque tout le mur. Et `PORTEE = 0` ? La boucle ne fait aucun tour : il ne reste que le centre.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <lire>` → [0.68.4. La fonction lire\(\) — lire une case](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <effacer>` → [0.16.3. La fonction effacer\(\) — effacer des cases](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <chaque>` → [0.29.3. La fonction chaque\(\) — toutes les n millisecondes](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1814 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	107 o	← la console, à l'allumage - la mise en route de la console
chaque_minuteur()	124 o	← main() - une fonction fournie par le langage
dessinerFeu()	54 o	← main() - ta fonction
allumer()	64 o	← exploser() - ta fonction
eteindre()	46 o	← eteindreTout() - ta fonction
exploser()	101 o	← main() - ta fonction
eteindreTout()	96 o	← main() - ta fonction
main()	923 o	← démarrage - ta fonction
VBlank	8 o	← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage	28 o	← image() - attend l'image suivante
AttendreVBlank	16 o	← démarrage - attend que l'écran soit libre
AttendreAcces	12 o	← EffacerCases, EcrireTexte - attend de pouvoir écrire une case
AdresseCase	10 o	← texte(), lire(), effacer(), poser() - trouve l'adresse d'une case de l'écran
EffacerCases	12 o	← effacer() - efface des cases
EcrireTexte	10 o	← texte() - écrit un texte, lettre par lettre
CopierTuiles	67 o	← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage - vide les deux cartes de l'écran
LireManette	37 o	← bouton() - lit les huit boutons
Tuiles	80 o	← CopierTuiles - 5 lettres sur 43
"0"	1 o	← texte() - un texte gravé
"#"	1 o	← texte() - un texte gravé
"#"	1 o	← texte() - un texte gravé
"."	1 o	← texte() - un texte gravé
"."	1 o	← texte() - un texte gravé

Lettres gravées : AOX.#

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

texte()	× 5	122 o sur place, dans dessinerFeu(), allumer(), main() + AdresseCase (10 o), EcrireTexte (10 o)
lire()	× 9	99 o sur place, dans allumer(), eteindre(), main() + AdresseCase (10 o)
effacer()	× 5	91 o sur place, dans eteindre(), main() + AdresseCase (10 o), EffacerCases (12 o)
poser()	× 3	38 o sur place, dans main() + AdresseCase (10 o)
bouton()	× 6	30 o sur place, dans main() + LireManette (37 o)
image()	× 1	3 o sur place, dans main() + AttendreImage (28 o)
images()	× 1	2 o sur place, dans chaque_minuteur()

Tes fonctions appelées :

allumer()	× 5	71 o sur place, dans exploser()
eteindre()	× 5	71 o sur place, dans eteindreTout()
chaque_minuteur()	× 4	43 o sur place, dans main()
dessinerFeu()	× 2	6 o sur place, dans main()
exploser()	× 1	3 o sur place, dans main()
eteindreTout()	× 1	3 o sur place, dans main()

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- UNE LETTRE QUI TIRE : une explosion plus grande ----
// Le 0.107, mais les bras de la croix ont une LONGUEUR : 2 cases de chaque
// côté au lieu d'une. Une boucle for allume les bras, case après case.
// Change PORTEE, et l'explosion grandit toute seule.
// ALPHABET[23] : le X. Le « # » est la tuile 42 de la police :
// espace = 0, A = 1 ... Z = 26, 0 = 27 ... 9 = 36, ! = 37, ? = 38,
// . = 39, - = 40, : = 41, # = 42.
// ALPHABET[23] : le X.
```

```
#include <texte> // écrit un texte à l'écran
#include <lire> // lit la tuile posée sur une case
#include <ALPHABET> // les lettres de la police : ALPHABET[0] est le A
#include <effacer> // efface des cases, ou tout le fond
#include <poser> // pose une tuile sur une case du fond
#include <chaque> // répond 1 toutes les n millisecondes
#include <bouton> // lit un bouton de la manette
```

2. UNE VARIABLE — AX

```
uint8_t ax = 2;           // le A : sa colonne...
```

3. UNE VARIABLE — AY

```
uint8_t ay = 8;           // ...et sa ligne
```

4. UNE VARIABLE — DX

```
uint8_t dx = 1;           // le sens où regarde le A
```

5. UNE VARIABLE — DY

```
uint8_t dy = 0;
```

6. UNE VARIABLE — NX

```
uint8_t nx = 0;           // une case d'ARRIVÉE, calculée avant de bouger
```

7. UNE VARIABLE — NY

```
uint8_t ny = 0;
```

8. UN TABLEAU — TX

```
uint8_t tx[3];            // les trois tirs : leurs colonnes...
```

9. UN TABLEAU — TY

```
uint8_t ty[3];            // ...leurs lignes...
```

10. UN TABLEAU — TDX

```
uint8_t tdx[3];           // ...et leurs sens
```

11. UN TABLEAU — TDY

```
uint8_t tdy[3];
```

12. UN TABLEAU — VOLE

```
uint8_t vole[3];          // vole[i] = 1 : le tir n° i est en vol
```

13. UNE VARIABLE — LANCE

```
uint8_t lance = 0;
```

14. UNE VARIABLE — FX

```
uint8_t fx = 0;           // NOUVEAU : la boule de feu : sa colonne...
```

15. UNE VARIABLE — FY

```
uint8_t fy = 0;           // ...sa ligne...
```

16. UNE VARIABLE — FDX

```
uint8_t fdx = 0;          // ...son sens...
```

17. UNE VARIABLE — FDY

```
uint8_t fdy = 0;
```

18. UNE VARIABLE — FEU

```
uint8_t feu = 0;           // ...1 si elle vole, 0 sinon (une seule à la fois)...
```

19. UNE VARIABLE — FLAMME

```
uint8_t flamme = 0;        // ...et son dessin : 0 = « 0 », 1 = « # »
```

20. UNE VARIABLE — BOUM

```
uint8_t boum = 0;          // NOUVEAU : 1 pendant l'explosion
```

21. UNE VARIABLE — EX

```
uint8_t ex = 0;            // le centre de l'explosion : sa colonne...
```

22. UNE VARIABLE — EY

```
uint8_t ey = 0;            // ...et sa ligne
```

23. UNE VARIABLE — DUREE

```
uint8_t duree = 0;         // les images qu'il reste avant de l'éteindre
```

24. UNE VARIABLE — PORTEE

```
uint8_t PORTEE = 2;        // NOUVEAU : la longueur des bras de la croix

// ---- NOUVEAU : dessiner la boule de feu, et changer son dessin ----
// Une fois « 0 », la fois suivante « # », puis « 0 »... : elle clignote.
// flamme = 1 - flamme : 1 - 0 = 1, et 1 - 1 = 0. Elle passe de l'un à l'autre.
```

25. UNE FONCTION — DESSINERFEU()

```
void dessinerFeu() {
    if (flamme == 0) {
        texte(fx, fy, "0");
    } else {
        texte(fx, fy, "#");
    }
    flamme = 1 - flamme;
}

// ---- NOUVEAU : allumer UNE case de l'explosion ----
// Dans l'écran (colonne 255 = sortie à gauche : refusée aussi), et seulement
// sur une case vide ou un X : le X est cassé, remplacé par le « # ».
// Le A, un tir... ne sont pas touchés.
```

26. UNE FONCTION — ALLUMER()

```
void allumer(uint8_t c, uint8_t l) {
    if (c < 20 && l < 17) {
        if (lire(c, l) == 0 || lire(c, l) == ALPHABET[23]) {
            texte(c, l, "#");
        }
    }
}

// ---- NOUVEAU : éteindre UNE case : seulement si c'est encore un « # » ----
```

27. UNE FONCTION — ETEINDRE()

```
void eteindre(uint8_t c, uint8_t l) {
    if (c < 20 && l < 17) {
        if (lire(c, l) == 42) {          // 42 : la tuile du « # »
            effacer(c, l, 1);
        }
    }
}

// ---- l'explosion, en croix, avec des bras de PORTEE cases ----
//          #                PORTEE = 2 :
//          #
//      # # # # #          le centre, et 2 cases dans chaque sens
//          #
//          #
// NOUVEAU : une boucle for. k vaut 1, puis 2 (jusqu'à PORTEE) :
//   k = 1 allume les quatre voisines, k = 2 les quatre suivantes.
```

28. UNE FONCTION — EXPLOSER()

```
void exploser() {
    allumer(ex, ey);                // le centre
    for (uint8_t k = 1; k <= PORTEE; k++) {
        allumer(ex + k, ey);        // à droite
        allumer(ex - k, ey);        // à gauche (sous 0 : 255, 254... refusés)
        allumer(ex, ey + k);        // en bas
        allumer(ex, ey - k);        // en haut
    }
    boum = 1;
    duree = 20;
}

// ---- éteindre la croix : la même boucle, avec eteindre() ----
```

29. UNE FONCTION — ETEINDRETOUT()

```
void eteindreTout() {
    eteindre(ex, ey);
    for (uint8_t k = 1; k <= PORTEE; k++) {
        eteindre(ex + k, ey);
        eteindre(ex - k, ey);
        eteindre(ex, ey + k);
        eteindre(ex, ey - k);
    }
    boum = 0;
}
```

30. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    for (uint8_t l = 4; l < 13; l++) {    // le mur : 9 X en colonne 12 (le 0.101)
        poser(12, l, ALPHABET[23]);
    }
    poser(ax, ay, ALPHABET[0]);

    while (true) {
        image();

        // ---- le A bouge et retient où il regarde ----
        if (chaque(150)) {
            nx = ax;
            ny = ay;
            if (bouton(DROITE)) {
                nx = ax + 1;
                dx = 1;
                dy = 0;
            } else if (bouton(GAUCHE)) {
                nx = ax - 1;
                dx = -1;
                dy = 0;
            } else if (bouton(BAS)) {
                ny = ay + 1;
                dx = 0;
                dy = 1;
            } else if (bouton(HAUT)) {
                ny = ay - 1;
                dx = 0;
                dy = -1;
            }
            if (nx < 20 && ny < 17) {
                if (lire(nx, ny) == 0) {
                    effacer(ax, ay, 1);
                    ax = nx;
                    ay = ny;
                    poser(ax, ay, ALPHABET[0]);
                }
            }
        }

        // ---- le bouton A lance un tir, au plus toutes les 250 ms ----
        if (chaque(250)) {
            if (bouton(A)) {
                nx = ax + dx;
                ny = ay + dy;
                // la case de départ doit être dans l'écran ET vide (le 0.101)
                if (nx < 20 && ny < 17 && lire(nx, ny) == 0) {
                    lance = 0;
                    for (uint8_t i = 0; i < 3; i++) {
                        if (lance == 0 && vole[i] == 0) {
                            vole[i] = 1;
                            tx[i] = nx;
                            ty[i] = ny;
                            tdx[i] = dx;
                            tdy[i] = dy;
                            texte(tx[i], ty[i], ".");
                            lance = 1;
                        }
                    }
                }
            }
        }
    }

    // ---- chaque tir avance, et regarde sa case d'arrivée (le 0.105) ----
```

```

if (chaque(100)) {
    for (uint8_t i = 0; i < 3; i++) {
        if (vole[i] == 1) {
            nx = tx[i] + tdx[i];
            ny = ty[i] + tdy[i];
            effacer(tx[i], ty[i], 1);
            if (nx >= 20 || ny >= 17) {
                vole[i] = 0;          // 1. hors de l'écran
            } else if (lire(nx, ny) == ALPHABET[23]) {
                effacer(nx, ny, 1);    // 2. un X : le tir le casse (le 0.105)...
                vole[i] = 0;          // ...et disparaît
            } else if (lire(nx, ny) != 0) {
                vole[i] = 0;          // 3. un autre tir : il s'écrase
            } else {
                tx[i] = nx;           // 4. une case vide : il y va
                ty[i] = ny;
                texte(tx[i], ty[i], ".");
            }
        }
    }
}

// ---- le bouton B lance la boule de feu (le 0.106, le 0.107) ----
// boum == 0 : pas de nouvelle boule tant que l'explosion brûle encore.
if (bouton(B) && feu == 0 && boum == 0) {
    nx = ax + dx;
    ny = ay + dy;
    if (nx < 20 && ny < 17 && lire(nx, ny) == 0) {
        feu = 1;
        fx = nx;
        fy = ny;
        fdx = dx;          // elle recopie le sens du A, comme un tir
        fdy = dy;
        dessinerFeu();
    }
}

// ---- la boule avance toutes les 200 ms (le 0.106) ----
if (chaque(200)) {
    if (feu == 1) {
        nx = fx + fdx;
        ny = fy + fdy;
        effacer(fx, fy, 1);        // elle quitte sa case
        if (nx >= 20 || ny >= 17) {
            feu = 0;              // 1. hors de l'écran : elle disparaît
        } else if (lire(nx, ny) != 0) {
            // 2. l'arrivée n'est pas vide (un X...) : BOUM ! (le 0.107)
            // L'explosion se centre sur la case touchée.
            feu = 0;
            ex = nx;
            ey = ny;
            exploser();
        } else {
            fx = nx;              // 3. une case vide : elle y va...
            fy = ny;
            dessinerFeu();        // ...avec l'autre dessin : elle clignote
        }
    }
}

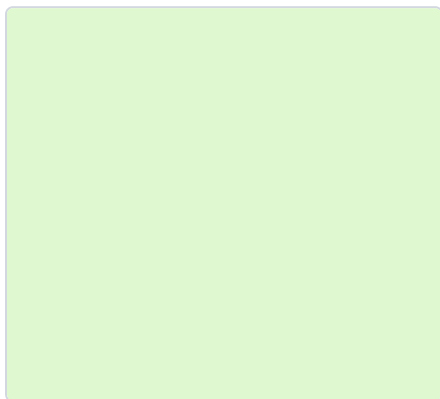
// ---- l'explosion brûle 20 images, puis s'éteint (le 0.107) ----
if (boum == 1) {
    duree = duree - 1;
    if (duree == 0) {
        eteindreTout();          // les « # » disparaissent : un trou reste
    }
}

```

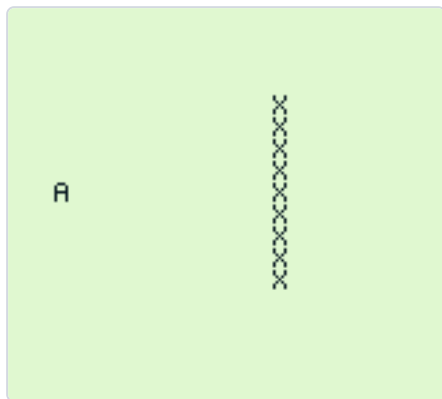
```
}  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



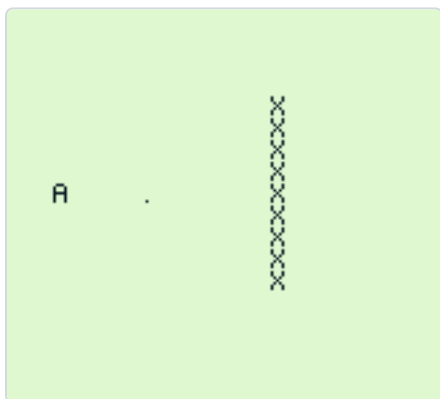
la toute première image — la console vient de démarrer — image 1



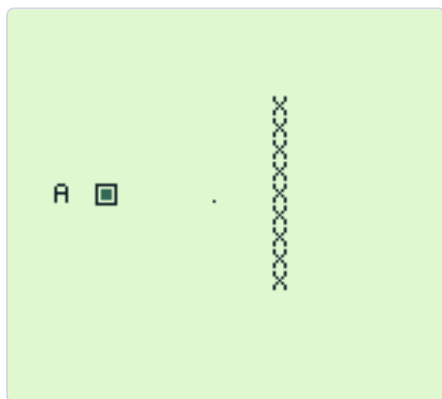
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

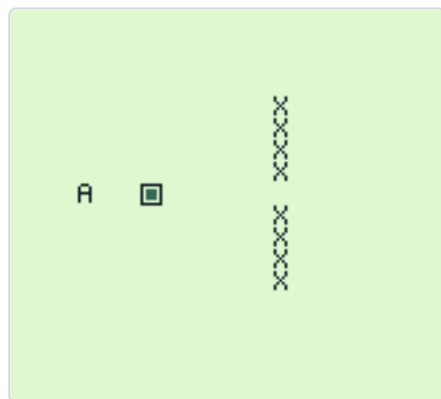
Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



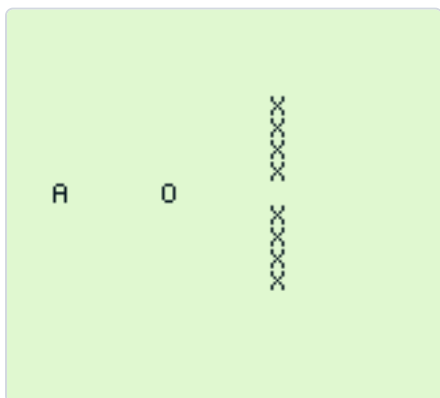
A — l'écran a changé



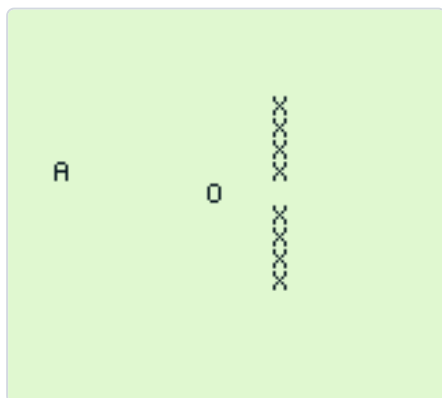
B — l'écran a changé



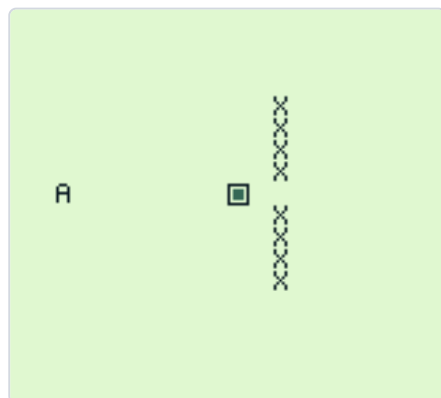
DROITE — l'écran a changé



GAUCHE — l'écran a changé



HAUT — l'écran a changé



BAS — l'écran a changé

5. Ce qu'on doit voir

La boule de feu explose en une grande croix de « # », deux cases dans chaque sens : cinq X cassés, des lignes 6 à 10.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ BOUM : une croix de « # » avec des bras de deux cases

✓ elle s'éteint : cinq X cassés, lignes 6 à 10

✓ les X des lignes 5 et 11 restent

Ensuite : **0.107.2. Tuile — dessiner une case** — chapitre 0 : Avant tout. · [gameboy3](#) — écrire une cartouche Game Boy en C++